



**COMSATS University Islamabad,
Sahiwal Campus.**

AI Campus Assistant

By

Muhammad Talha

CIIT/FA22-BCS-081/SWL

Muhammad Zaid Zia

CIIT/FA22-BCS-089/SWL

Supervisor

Mr. M. Arslan Sarwar

Bachelor of Science in Computer Science (2022-2026)



**COMSATS University Islamabad,
Sahiwal Campus.**

AI Campus Assistant

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Talha

CIIT/FA22-BCS-081/SWL

Muhammad Zaid Zia

CIIT/FA22-BCS-089/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Talha

Muhammad Zaid Zia

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**AI Campus Assistant**” was developed by **Muhammad Talha (CIIT/FA22-BCS-081/SWL)** and **Muhammad Zaid Zia (CIIT/FA22-BCS-089/SWL)** under the supervision of “**Mr. M. Arslan Sarwar**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. M. Arslan Sarwar

Lecturer

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Ansar Munir Shah

Assistant Professor

Department of Computer Science

University of Southern Punjab, Multan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

AI Campus Assistant is an intelligent chatbot application that caters specifically to university students and faculty. Unlike general-purpose chat systems, AI Campus Assistant is designed to address the specific needs of a university environment by providing accurate and instant information related to admissions, programs, fee structures, academic schedules, student queries, campus facilities, and general guidelines.

This system allows users to interact with a chatbot to get reliable answers based on official university data. Students can ask questions related to their daily academic needs such as finding their classes, understanding timetables, calculating CGPA, and resolving other university-related queries. Instead of relying on manual support or searching through multiple sources, users can simply ask questions and receive quick responses.

The application provides a simple and user-friendly interface that allows users of all technical levels to easily navigate and interact with the system. For administrators, it offers a structured way to manage and update university data, ensuring that the information provided to users remains accurate and up to date.

The system is built using a Retrieval-Augmented Generation (RAG) approach, where relevant information is retrieved from a structured knowledge base and then used to generate responses through an AI model. This approach improves the accuracy and reliability of the chatbot responses.

The development of the application follows an agile model, allowing features to be implemented and improved in phases. The system is designed based on both functional and non-functional requirements to ensure usability, performance, and security.

Testing and evaluation of the system include unit testing, integration testing, and system testing to verify that all components are functioning correctly. User acceptance testing is also conducted to gather feedback and enhance the overall usability of the application.

In summary, **AI Campus Assistant** provides a centralized and efficient platform for accessing university-related information. It improves communication, reduces dependency on manual support, and enhances the overall university experience by providing quick, accurate, and reliable assistance to users.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. M. Arslan Sarwar**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Talha

Muhammad Zaid Zia

Abbreviations

API	Application Programming Interface
RAG	Retrieval-Augmented Generation
SRS	Software Require Specification
SDD	Software Design Description
GUI	Graphical User Interface
IDEs	Integrated development environments
LLM	Large Language Model
DB	Database
AI	Artificial Intelligence

Table of Contents

1	Introduction	1
1.1	Brief Overview	2
1.2	Relevance to Course Modules	2
1.3	Project Background	3
1.4	Methodology and Software Lifecycle	4
1.4.1	Rationale behind Selected Agile Model	4
2	Problem Definition	5
2.1	Problem Statement	5
2.2	Deliverables and Development Requirements	5
2.3	Current System	7
3	Requirement Analysis	8
3.1	Use Case Diagrams	8
3.2	Use Case Descriptions	8
3.3	Functional Requirements	11
3.3.1	Ask Query	11
3.3.2	Receive Answer	11
3.3.3	Retrieve Relevant Documents	11
3.3.4	User Interaction (Chat Module)	11
3.3.5	Feedback System	11
3.3.6	Query History Management	11
3.3.7	Admin Notification on Failed Queries	12
3.3.8	Document Management (Admin Module)	12
3.4	Nonfunctional Requirements	12
3.4.1	Performance	12
3.4.2	Scalability	12
3.4.3	Security	12
3.4.4	Usability	12
3.4.5	Maintainability	12
3.4.6	Reliability	13
3.4.7	Availability	13
4	Design and Architecture	14

4.1	System Architecture	14
4.2	Process Flow/Representation	15
4.3	Sequence Diagram.....	15
4.4	Class Diagram	18
5	Implementation	20
5.1	Admin Module	20
5.2	Student Module Implementation.....	20
5.3	Real Time Chat and Messaging Algorithm.....	21
5.4	Security and Data Backend Implementation.....	21
5.5	Security and Data Handling	21
5.6	User interface	21
6	Testing	32
6.1	Unit Testing.....	32
6.2	Manual Testing.....	32
6.3	Functional Testing.....	33
6.4	Integration Testing	34
7	Conclusion and Future Work	35
7.1	Conclusion.....	35
7.2	Future Work	35
	References	37

List of Figures

Figure 3-1 AI campus Assistant Use Case.....	8
Figure 4-1 AI Campus Assistant architecture design	14
Figure 4-2 AI Campus Assistant Process flow	15
Figure 4-3 AI Campus Assistant knowledge-base Sequence	18
Figure 4-4 AI Campus Assistant Class Diagram	19
Figure 5-1 AI Campus Assistant Login screen	22
Figure 5-2 AI Campus Assistant Account Screen	23
Figure 5-3 AI Campus Assistant Chat Screen	24
Figure 5-4 AI Campus Assistant Comsats plus Screen	25
Figure 5-5 AI Campus Assistant Course grade calculator Screen	26
Figure 5-6 AI Campus Assistant semester GPA calculator Screen	27
Figure 5-7 AI Campus Assistant CGPA calculator Screen	28
Figure 5-8 AI Campus Assistant GPA planning Screen.....	29
Figure 5-9 AI Campus Assistant FAQs Screen	30
Figure 5-10 AI Campus Assistant Settings Screen	31

List of Tables

Table 3.1 Use case description for Ask Query	8
Table 3.2 Use case description for receive answer	9
Table 3.3 Use case description for Notify Admin on Failed Queries	9
Table 3.4 Use case description for provide feedback	9
Table 3.5 Use case description for view query history	10
Table 3.6 Use case description for upload/update documents	10
Table 6.1 Unit Testing	32
Table 6.2 ManualTesting	33
Table 6.3 Functional Testing	33
Table 6.4 Integration testing	34

Chapter 1

Introduction

1 Introduction

AI Campus Assistant is an intelligent addition to the university ecosystem, designed to address the common challenges faced by students, faculty, and visitors during their academic journey. This application is specifically developed for the COMSATS University Islamabad (CUI), Sahiwal Campus, providing a smart and reliable way to access university-related information. It is tailored to meet the specific needs of users by offering instant responses to queries related to admissions, academic schedules, fee structures, campus facilities, and general guidelines.

The core strength of the system lies in its use of **Retrieval-Augmented Generation (RAG)** combined with a **Large Language Model (LLM)**. When a user submits a query, the system first retrieves the most relevant information from a structured knowledge base stored in a vector database. This retrieved information is then passed to the LLM, which generates a clear, context-aware, and human-like response. This approach ensures that the chatbot does not generate random answers but provides responses strictly based on verified university data.

Furthermore, the AI Campus Assistant also serves as an efficient tool for university administration. Information is managed through Google Sheets, allowing administrators to easily update and maintain official data. This data is processed into chunks and converted into embeddings, which are stored in the vector database for efficient retrieval. As a result, all responses provided to users remain accurate, consistent, and up-to-date.

Currently, there is no dedicated intelligent assistant available for students of COMSATS University Islamabad (CUI), Sahiwal Campus that provides instant and reliable university-specific information. Students often face difficulties in finding accurate information quickly, especially during admissions or academic activities. The AI Campus Assistant project aims to solve this problem by providing a centralized and intelligent solution for handling queries.

The application also focuses on user-friendliness, ensuring that individuals with different technical backgrounds can easily interact with the system. Its simple chat-based interface allows users to communicate naturally without requiring any technical knowledge.

In essence, AI Campus Assistant is designed to fulfill the modern requirements of university life by providing a smart, efficient, and reliable platform for accessing information. It enhances user experience, improves communication, and supports students in their academic activities.

1.1 Brief Overview

The AI Campus Assistant project aims to develop a mobile application exclusively for COMSATS University Islamabad, Sahiwal Campus users to provide instant access to university-related information. The application is developed using Flutter for the frontend and FastAPI for the backend, along with an AI-based Retrieval-Augmented Generation (RAG) system for intelligent response generation [1].

AI Campus Assistant is a unique system that focuses on solving information accessibility problems within the university. Instead of general communication, it provides a knowledge-based chatbot that answers user queries accurately using official university data. The system retrieves relevant information from a structured database and generates context-aware responses, making it a reliable and efficient solution.

1.2 Relevance to Course Modules

The development of AI Campus Assistant integrates various concepts and skills learned throughout the software engineering course.

i. Technical Skills

Application of programming languages, frameworks, and tools such as Flutter, FastAPI, and AI-based technologies for building the system.

ii. Software Engineering Principles

Software engineering principles are applied throughout the project lifecycle, including requirement gathering, system analysis, design, implementation, testing, and maintenance. A structured development approach ensures that the system is reliable, scalable, and maintainable. Each phase is carefully executed to reduce errors and improve software quality [2].

iii. Database Design

Database concepts are applied to manage and retrieve data efficiently. A vector database such as **ChromaDB** is used to store embeddings, allowing semantic search and intelligent information retrieval. Structured data is also maintained for system operations, ensuring fast and accurate responses to user queries.

iv. System Architecture

The system follows a modular architecture consisting of three main layers: frontend, backend, and AI processing unit. The frontend handles user interaction, the backend manages logic and API communication, and the AI module processes natural language queries. This separation of concerns improves scalability, maintainability, and system performance.

v. UI/UX Design

The user interface of AI campus Assistant is designed to be simple, clean, and intuitive so that users can easily interact with the chatbot. Focus is placed on usability, ensuring that students can quickly access information without technical difficulty. A responsive design approach is used to support different devices and screen sizes.

vi. Project Management

Project management techniques are applied in the development of AI campus Assistant to ensure smooth development and timely completion. Tasks are organized into phases, and progress is monitored regularly. Planning tools and iterative development cycles help in tracking milestones and managing workload efficiently.

vii. Artificial Intelligence & NLP

Artificial Intelligence plays a central role in the **AI Campus Assistant** project. Techniques such as Natural Language Processing (NLP), embeddings, and Retrieval-Augmented Generation (RAG) are used to understand user queries and generate accurate responses. These technologies enable the system to provide intelligent, context-aware answers based on stored academic and campus-related knowledge.

1.3 Project Background

The **AI Campus Assistant** was developed in response to common challenges faced by university students in accessing accurate and timely information. In many academic institutions, students rely on manual methods such as visiting administrative offices, checking notice boards, or asking peers for information.

These traditional methods are often inefficient, time-consuming, and sometimes unreliable. Students face difficulties in obtaining quick answers related to admissions, exam schedules, fee structures, course information, and campus facilities.

To address these issues, the AI Campus Assistant provides a centralized and intelligent platform where students can directly ask questions and receive instant, accurate responses. The system reduces dependency on manual processes and ensures that information is accessible at any time. By integrating AI and structured knowledge sources, the system improves communication efficiency and enhances the overall student experience.

1.4 Methodology and Software Lifecycle

AI Campus Assistant is developed as a dynamic software system with evolving requirements. Due to the nature of AI-based applications, user needs and system behavior may change over time. Therefore, a flexible development approach is required.

For this purpose, the **Agile methodology** is selected. Agile supports iterative development, continuous improvement, and active user involvement throughout the project lifecycle.

1.4.1 Rationale behind Selected Agile Model

Agile is a modern software development methodology that emphasizes flexibility, collaboration, and incremental progress. It is particularly suitable for AI-based systems where requirements are not always fixed at the beginning and may evolve during development.

In the AI Campus Assistant project, Agile allows the system to be developed in small, manageable increments. Each iteration adds new features or improves existing functionality based on testing and feedback. This ensures continuous enhancement of system quality [3].

Agile is based on the following principles:

- Iterative development:** The system is developed step by step in multiple cycles. Each iteration focuses on adding or improving specific features, allowing gradual system growth.
- Incremental delivery:** Functional components of the system are delivered in phases. This ensures that a working version of the application is always available during development.
- Continuous feedback:** Regular feedback is collected from users and stakeholders. This feedback is used to refine system features, improve accuracy, and enhance usability.
- Adaptive planning:** Project requirements are flexible and can be adjusted based on new insights or changing needs. This adaptability is essential for AI-based systems where improvements are continuous.

Chapter 2

Problem Definition

2 Problem Definition

This chapter discusses the specific problem addressed by the **AI Campus Assistant** project and highlights the expected outcomes of the proposed solution.

2.1 Problem Statement

The **AI Campus Assistant** project aims to address the common problem faced by university students in accessing accurate, timely, and reliable campus-related information. Students often rely on traditional methods such as visiting administrative offices, browsing multiple web pages, or asking peers, which are time-consuming and sometimes inefficient.

Currently, there is no centralized intelligent system within the university that can instantly respond to student queries related to admissions, schedules, fees, courses, and campus services. Existing systems also lack the ability to provide context-aware and accurate answers based on official data sources.

To solve this issue, the **AI Campus Assistant** uses **Retrieval-Augmented Generation (RAG)** combined with **LLM**, which combines information retrieval with AI-based response generation. This approach allows the system to fetch relevant information from a knowledge base and generate accurate, context-aware responses. As a result, the system ensures reliability, reduces misinformation, and improves the efficiency of information access for students.

2.2 Deliverables and Development Requirements

The deliverables and development requirements of the **AI Campus Assistant** project include the following components:

- **User Interface (UI) Design:** Develop a simple, intuitive, and user-friendly chat interface that allows students to interact easily with the system. The interface should be responsive and accessible across multiple devices.
- **AI-Based Chat Functionality:** Implement an intelligent chatbot capable of understanding natural language queries and generating accurate responses using AI techniques.
- **User Interaction Module:** Allow users to input queries and receive instant responses in a conversational format, improving usability and engagement.

- **Knowledge Base Integration:** Develop a structured knowledge base that stores university-related information such as policies, schedules, FAQs, and academic details. This data is used by the AI system to retrieve relevant information and provide accurate answers to user queries.
- **Vector Database (ChromaDB):** Implement a vector database such as **ChromaDB** to store embeddings generated from textual data. This enables semantic search, allowing the system to find relevant information based on meaning rather than exact keywords, which improves the accuracy of responses.. .
- **Backend Development (FastAPI):** Develop a robust backend system using FastAPI to handle API requests, process queries, and connect different system components.
- **Security and Data Privacy:** Ensure the protection of user queries and system data through proper security mechanisms and controlled access.
- **Scalability:** Design the system to handle increasing numbers of users and data efficiently through scalable architecture.
- **Cross-Platform Compatibility:** Ensure that the AI Campus Assistant can be accessed on different devices such as mobile phones, tablets, and desktops.
- **Documentation:** Create user manuals and developer documentation to aid users and future development efforts.
- **Testing and Quality Assurance:** Perform unit testing, integration testing, and user testing to identify fix and errors. This ensures the system reliability and accuracy of AI-generated responses.
- **User Training and Support:** Develop user-training materials and provide support channels to assist users with the application.
- **Regular Updates and Maintenance:** Plan for continuous updates to improve system performance and accuracy. This includes updating the knowledge base, improving AI responses, and fixing bugs to keep the system up to date with changing requirements.

These deliverables and development requirements collectively contribute to the successful development and continuous improvement of the AI Campus Assistant, aligning with the project's objective of providing accurate, efficient, and intelligent access to university-related information for students.”

2.3 Current System

Currently, there is no dedicated AI-based system within the university that provides instant and intelligent responses to student queries. Students depend on manual methods such as visiting offices, checking notice boards, or searching through multiple online sources.

These methods are often inefficient, time-consuming, and may not always provide accurate or up-to-date information. Additionally, the lack of a centralized system leads to inconsistent communication and difficulty in accessing important academic and administrative details.

The absence of such a system highlights the need for the **AI Campus Assistant**, which aims to provide a centralized, intelligent, and efficient solution for handling student queries and improving information accessibility.

Chapter 3

Requirement Analysis

3 Requirement Analysis

3.1 Use Case Diagrams

In this Use case diagrams, we show the behavior of our system. This enable you to visualize the different types of roles in a system and how those roles interact with the system.

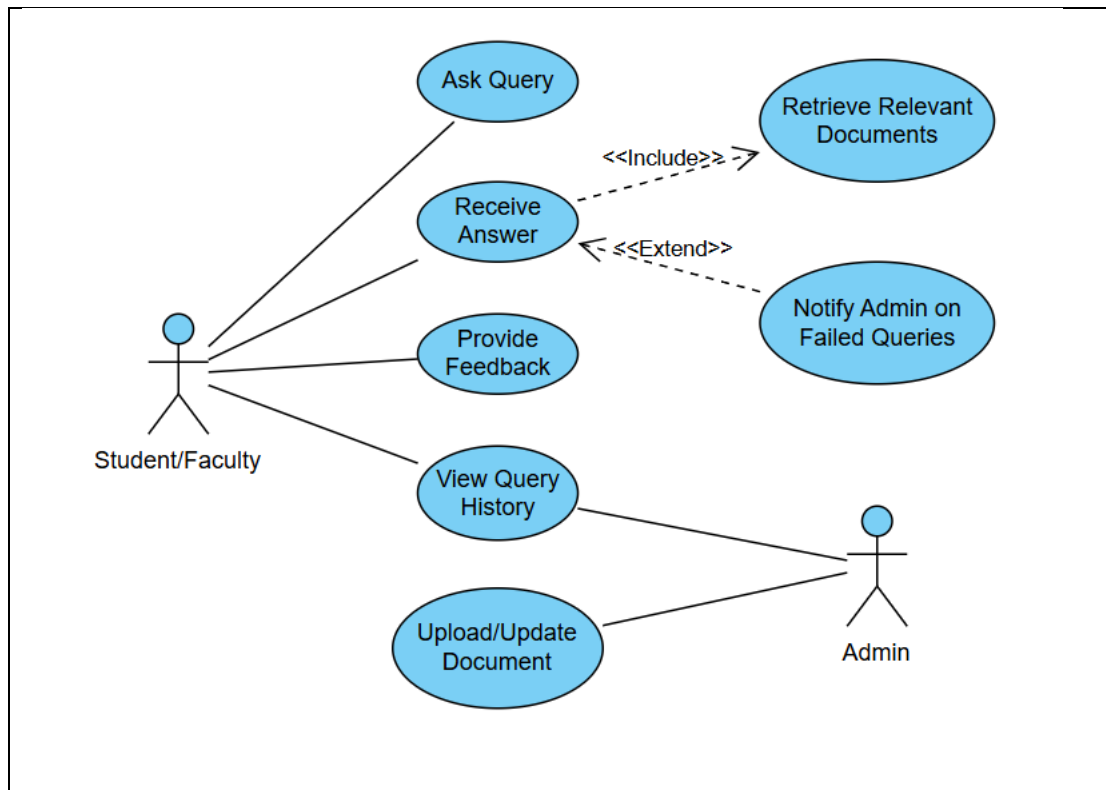


Figure 3-1 AI campus Assistant Use Case

3.2 Use Case Descriptions

The table below indicate comprehensive use case descriptions

Table 3.1 Use case description for Ask query

Use Case Id	UC-01
Use Case Name	Ask Query
Actors	Student/Faculty

Description	Actor submits a question regarding admissions, programs, fees, schedules, or campus facilities etc.
Trigger	User enters a query
Preconditions	User is logged in and connected to the internet.
Post Condition	Query sent to system for processing.

Table 3.2 Use case description for user receive answer

Use Case Id	UC-02
Use Case Name	Receive Answer
Actors	Student/Faculty
Description	System provides a context-aware response generated by RAG + LLM.
Trigger	Query is submitted by user.
Preconditions	Query has been validated, and relevant documents exist in the database.
Post Condition	User receives the answer displayed in the app.

Table 3.3 Use case description for Notify Admin on Failed queries

Use Case Id	UC-03
Use Case Name	Notify Admin on Failed Queries
Actors	Student/Faculty
Description	Send notification to admin if a query cannot be answered or show incorrect results.
Trigger	Triggered by Receive Answer.
Preconditions	Query cannot be answered by system or show incorrect results.
Post Condition	Admin is notified.

Table 3.4 Use case description for provide feedback

Use Case Id	UC-04
Use Case Name	Provide Feedback

Actors	Student/Faculty
Description	User provides feedback on system responses.
Trigger	User selects feedback option.
Preconditions	Requires successful query and response.
Post Condition	Feedback recorded in system.

Table 3.5 Use case description for view query history

Use Case Id	UC-05
Use Case Name	View Query History
Actors	Student/Faculty/Admin
Description	Actor reviews past queries and answers.
Trigger	User selects history option.
Preconditions	User has previous queries.
Post Condition	History displayed.

Table 3.6 Use case description for upload/update documents

Use Case Id	UC-06
Use Case Name	Upload/Update Documents
Actors	Admin
Description	Admin uploads or updates official university documents.
Trigger	Admin wants to add/update docs.
Preconditions	Admin is authenticated.
Post Condition	Documents stored and indexed for retrieval.

3.3 Functional Requirements

Functional requirements are the core features and functions that the system must provide to enable users to accomplish their tasks effectively. These requirements define how the system behaves and what it should do. It is important to clearly specify these requirements so that both the development team and stakeholders have a complete understanding of the system.

This system will have the following modules:

3.3.1 Ask Query

Users should be able to submit queries related to university information through a chat interface. The system should allow students, faculty, and visitors to enter questions in natural language.

3.3.2 Receive Answer

The system should generate accurate and context-aware responses to user queries using AI techniques. Responses should be based on retrieved information from the knowledge base using the RAG approach.

3.3.3 Retrieve Relevant Documents

The system should retrieve the most relevant documents from the vector database before generating responses. This ensures that answers are based on accurate and meaningful information.

3.3.4 User Interaction (Chat Module)

The system should provide a conversational interface where users can interact continuously, similar to real-time chat, improving usability and engagement.

3.3.5 Feedback System

Users should be able to provide feedback on the responses received, indicating whether the answer was helpful or not. This will help improve system performance over time.

3.3.6 Query History Management

Users should be able to view their previous queries and system responses. This feature allows users to revisit past information when needed.

3.3.7 Admin Notification on Failed Queries

The system should notify the admin when a query cannot be answered correctly or when the response is marked incorrect by the user. This helps in improving the knowledge base.

3.3.8 Document Management (Admin Module)

Admin should be able to upload, update, and manage official university documents (such as PDFs, DOCX files, and text data) to maintain an updated knowledge base for the system.

3.4 Nonfunctional Requirements

Nonfunctional requirements are those that establish parameters that may be used to evaluate the performance of a system. Nonfunctional requirements are the limitations to which the system will work.

3.4.1 Performance

The system shall respond to user queries within 5 seconds under normal load. This ensures smooth and responsive user experience while interacting with the AI Campus Assistant.

3.4.2 Scalability

The system shall be able to handle concurrent users without performance degradation. This ensures that multiple users can interact with the chatbot simultaneously without delays..

3.4.3 Security

All communication between the mobile application and the backend shall use HTTPS to protect sensitive data. Access to administrative features will be restricted to authorized personnel.

3.4.4 Usability

The application shall provide an intuitive and easy-to-use interface, ensuring that new and existing students can interact with the assistant with minimal guidance. Feedback mechanisms should be simple and accessible.

3.4.5 Maintainability

The system shall allow administrators to update or add new documents easily, and feedback collected from users shall be incorporated into periodic updates to improve accuracy and functionality.

3.4.6 Reliability

The AI Campus Assistant shall provide consistent and accurate responses by using only official university documents as its knowledge base. The RAG pipeline must reliably retrieve relevant documents for response generation.

3.4.7 Availability

The **AI Campus Assistant** system should ensure high availability so that users can access it anytime without interruption. It should be operational 24/7 to provide continuous access to university-related information. The backend, RAG pipeline, and database should be hosted on reliable infrastructure to minimize downtime. In case of failures, quick recovery mechanisms should be in place to restore services as soon as possible.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

The Architecture diagram represents the structure of the system explains the working of the system and describes the components of the AI Campus Assistant , their relationships, and how they interact with each other, with this architecture. The architecture diagram shows the overall design of the AI Campus Assistant system. The admin manages university data using Google Sheets. This data is processed into smaller chunks to make it suitable for searching. These chunks are converted into embeddings using a sentence transformer model. The embeddings are stored in ChromaDB, which works as a vector database. The FastAPI backend acts as the main bridge between all components. The student interacts through a Flutter application by sending queries. The backend retrieves relevant data from ChromaDB and sends it to the LLM. Finally, the LLM generates an answer and displays it in the app [4].

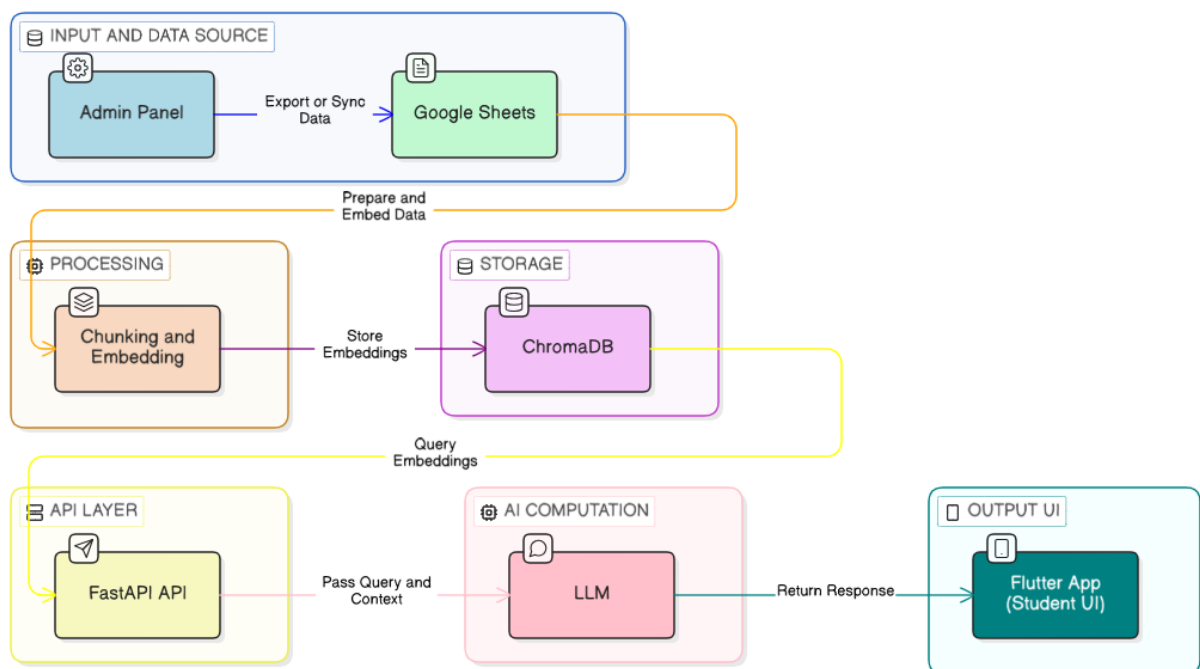


Figure 4-1 AI Campus Assistant architecture design

4.2 Process Flow/Representation

The Process Flow Representation illustrates the step-by-step sequence of actions and interactions between different components in the AI campus Assistant. The process flow diagram shows how the AI Campus Assistant handles data and user queries. The admin first updates university information in Google Sheets, which is then converted into chunks, embedded, and stored in ChromaDB. When a student logs in and submits a query, the system retrieves relevant data chunks from the database [5].

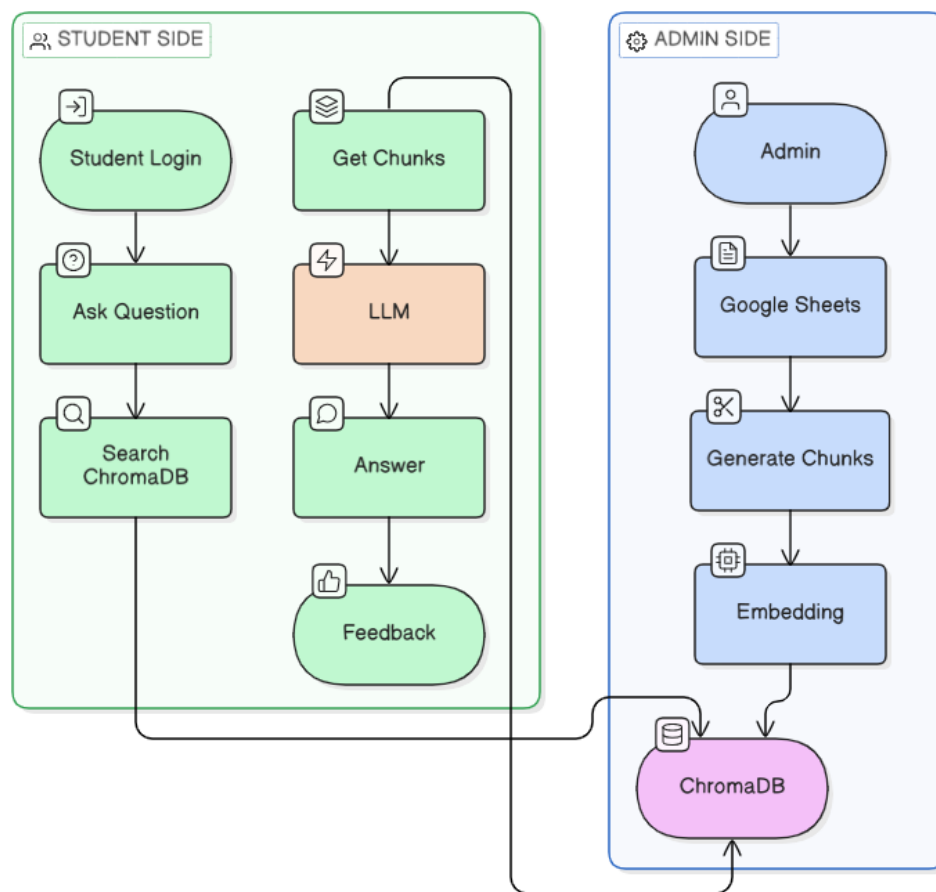


Figure 4-2 AI Campus Assistant Process flow

4.3 Sequence Diagram

The **sequence diagram** represents the interaction between different components of a system in a time-ordered sequence. It shows how messages or data flow step by step between actors like users, frontend, backend, database, and services. It clearly illustrates the order of operations from

the initial request to the final response. Figure shows how the system will work through sequence diagrams, task after task sequence wise. It gives a complete description of the working of AI campus Assistant [6].

Module 1: Student Module

The student module allows users to log in their university email and interact with the chatbot.

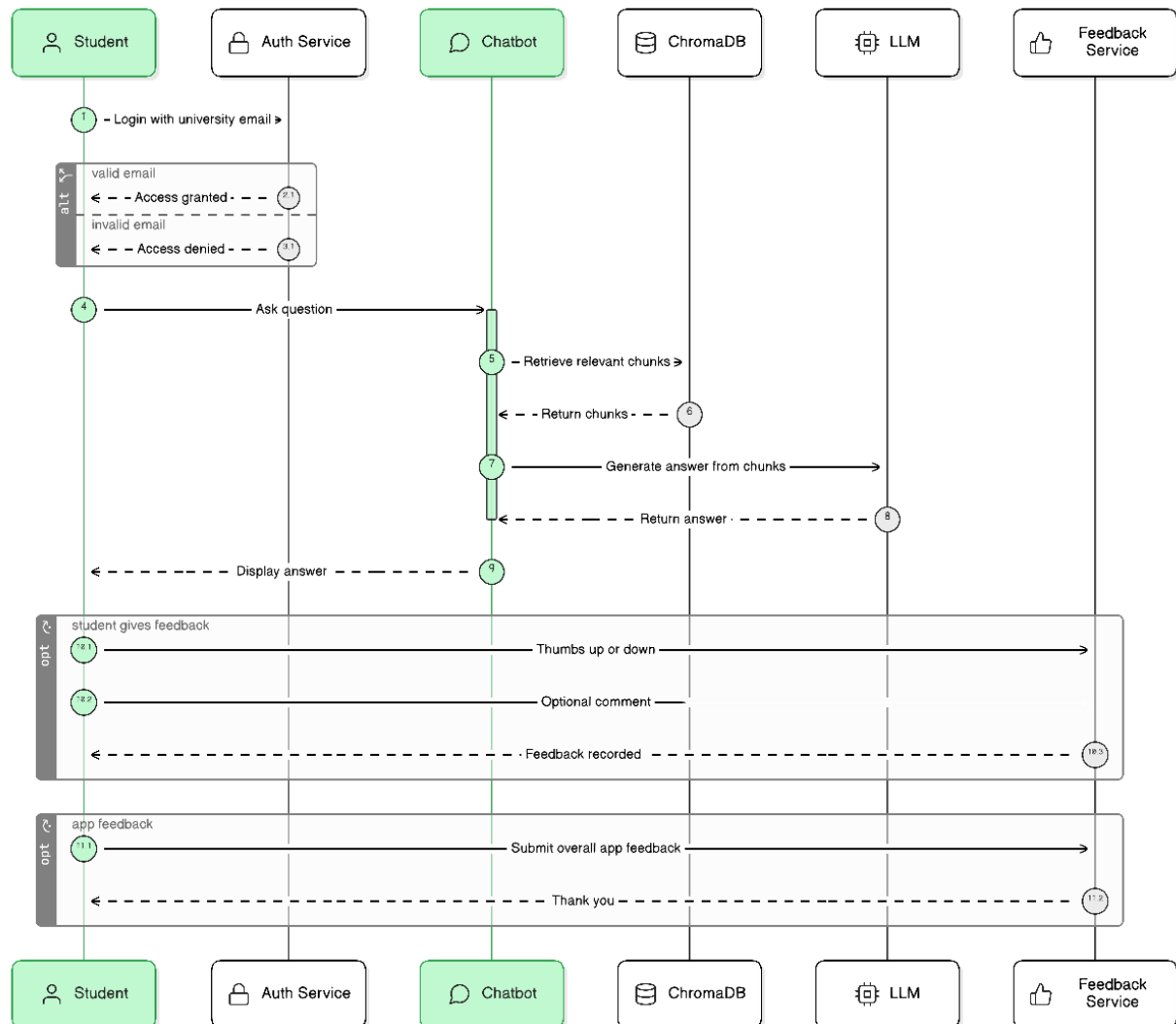


Figure 4.3 AI Campus Assistant Student Sequence

Module 2: Admin Module

The admin module allows the administrator to manage and update university data through Google Sheets. The updated data is processed into smaller chunks for efficient handling. These chunks are then converted into embeddings using a sentence transformer model. The embeddings represent the semantic meaning of the data in numerical form. After processing, the embeddings are stored in ChromaDB. This stored knowledge is later used for accurate

query retrieval in the system.

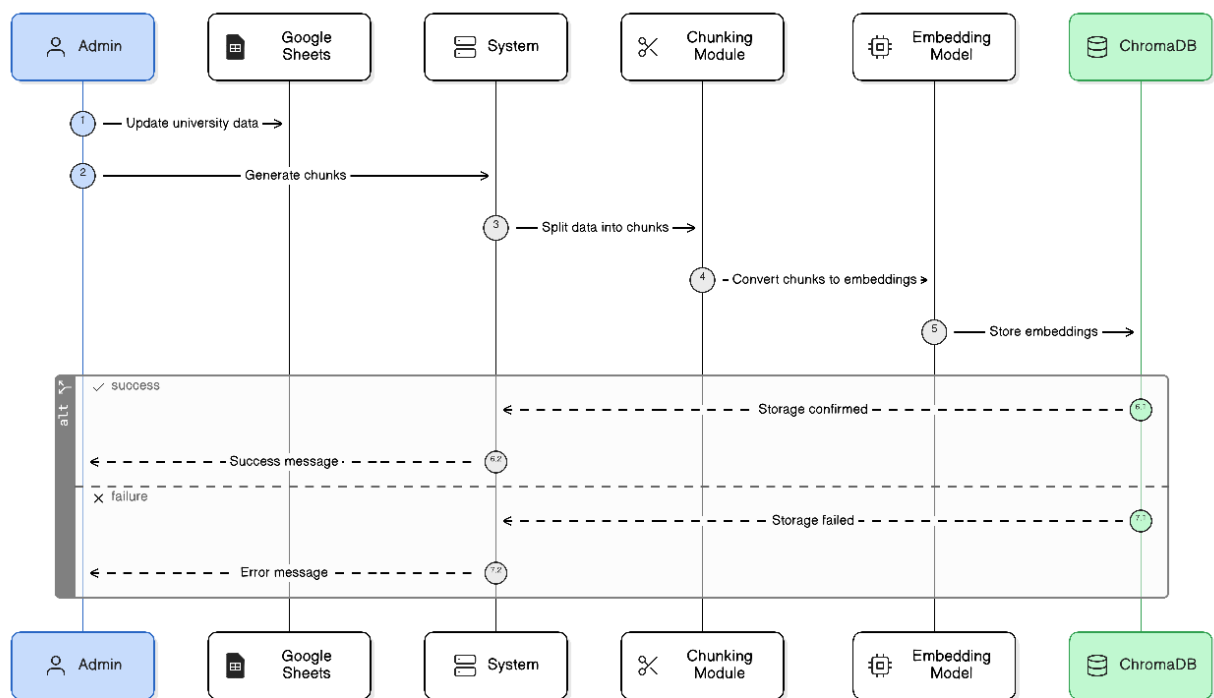


Figure 4.4 AI Campus Assistant Admin Sequence Diagram

Module 3: Knowledge Base Retrieval Module Module

The knowledge base retrieval module receives the student's query from the FastAPI backend. It processes the query and searches ChromaDB for the most relevant stored embeddings. The system identifies and retrieves the top matching data chunks based on similarity. These chunks provide the necessary context related to the question. The retrieved information is then sent to the LLM for processing. Finally, the LLM generates an accurate response which is returned to the student.

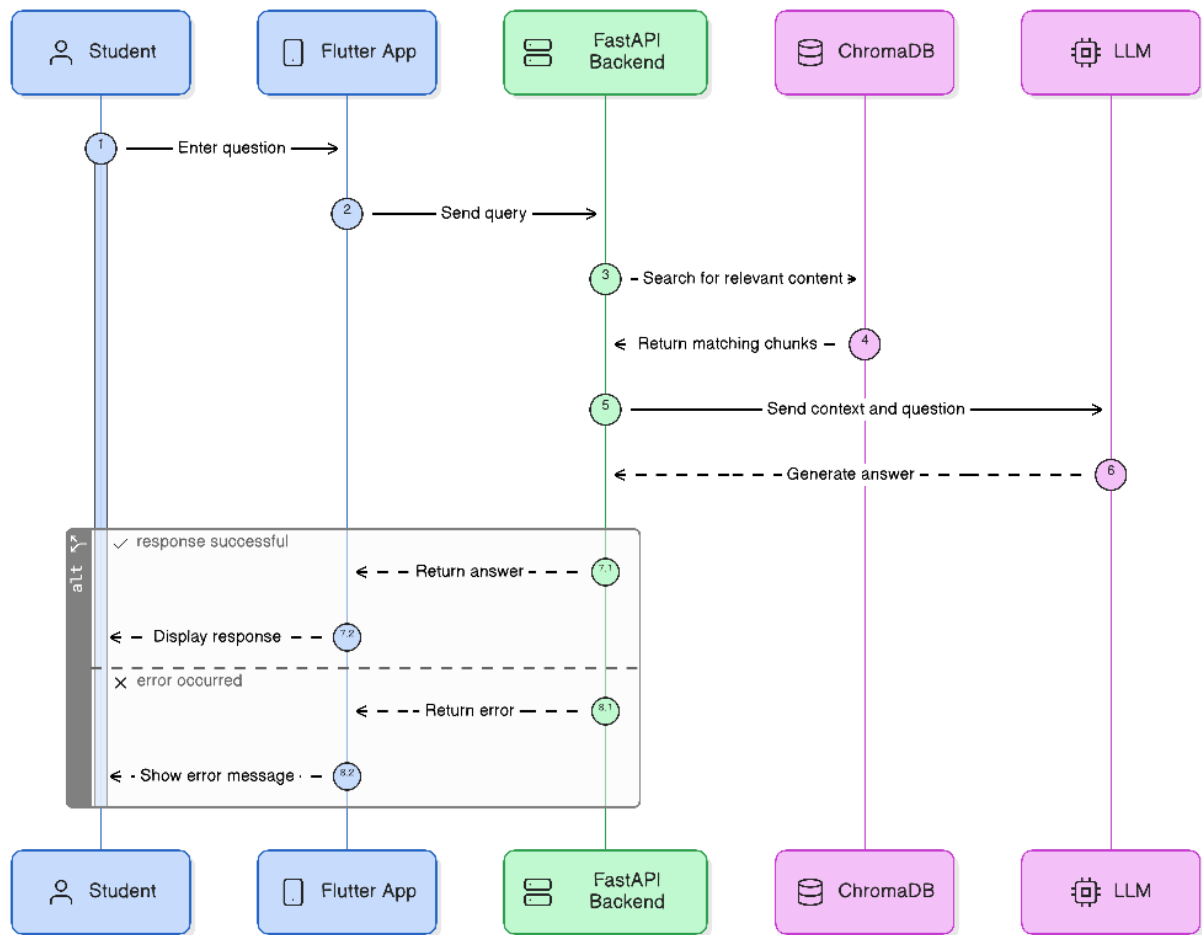


Figure 4-3 AI Campus Assistant knowledge-base Sequence

4.4 Class Diagram

This class diagram is a graphical representation of the system's classes, their attributes, and the relationships between them. It elaborates structure of the AI Campus Assistant and shows how the different classes in the system are related to each other. The class diagram represents the main classes of the AI Campus Assistant system and their responsibilities. The Admin class manages and updates university data. The Chunk Processor splits the data into smaller parts, which are then converted into embeddings by the Embedding Model. These embeddings are stored and retrieved from ChromaDB. The FastAPI class handles communication between the system components, while the LLM generates responses. Finally, the Student App class allows users to send queries and view answers.

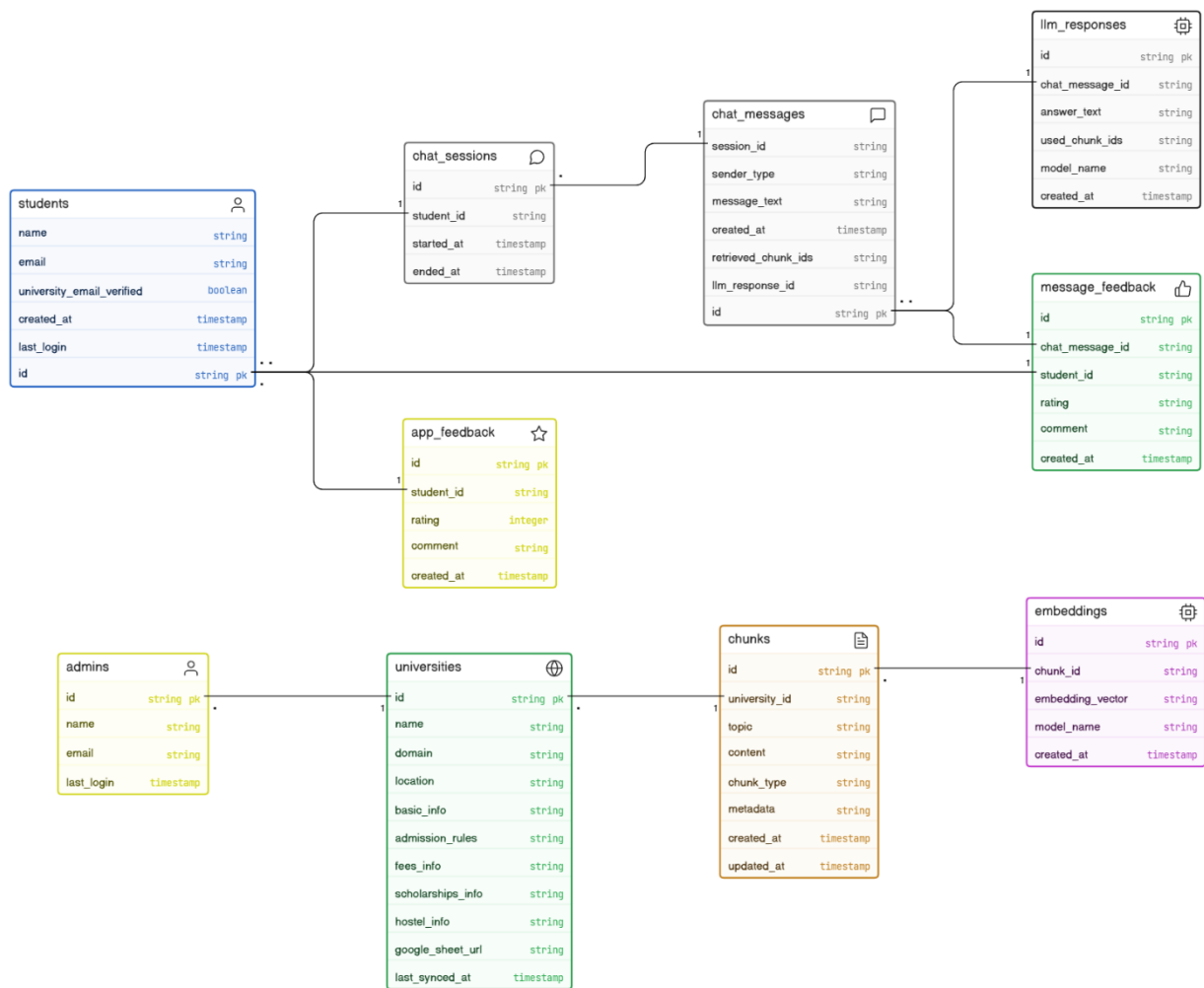


Figure 4-4 AI Campus Assistant Class Diagram

Chapter 5

Implementation

5 Implementation

In the implementation phase of the AI Campus Assistant system, we designed and developed the core modules to ensure efficient data management, accurate information retrieval, and smooth interaction between students and the system. The implementation is divided into Admin Module, Student Module, Knowledge Base Retrieval System, and supporting backend services.

5.1 Admin Module

The Admin Module is responsible for managing and maintaining university-related data. An Admin Panel is developed to allow easy updating of information such as admissions, fees, scholarships, hostel details, and academic policies using Google Sheets.

When the admin updates data, a **chunk generation process** is triggered. This process converts large text into smaller meaningful chunks such as paragraphs and FAQ pairs to make retrieval more efficient for the RAG system.

These chunks are then transformed into **embeddings** using the Sentence Transformer model (all-MiniLM-L6-v2), which converts text into numerical vectors representing semantic meaning.

Finally, these embeddings are stored in **ChromaDB**, a vector database that supports fast similarity search. An additional **search testing feature** is also implemented in the admin panel to verify whether correct data is being retrieved before deployment [7].

5.2 Student Module Implementation

The Student Module provides an interface for users to interact with the chatbot. Students must first authenticate using their university email, ensuring secure and authorized access.

After login, students can enter their queries in a **text-based chat interface** developed in Flutter. Voice input is not supported in this version.

The query is sent to the backend, where the **RAG (Retrieval-Augmented Generation) pipeline** is executed. The system retrieves relevant chunks from ChromaDB and passes them to the LLM.

The LLM generates a response strictly based on the retrieved university data, ensuring accurate and reliable answers. Students can also provide **feedback (thumbs up/down and comments)** on responses, which helps improve system quality.

5.3 Real Time Chat and Messaging Algorithm

The retrieval system is a core component of the application. When a student submits a query, it is first processed by the FastAPI backend. The system then searches ChromaDB to find the most relevant embeddings based on semantic similarity.[8]

5.4 Security and Data Backend Implementation

The backend of the system is developed using **FastAPI (Python)**, which acts as the central communication layer between the frontend, database, and AI models.

It handles API requests from the Flutter application, processes user queries, interacts with ChromaDB, and integrates the LLM for response generation. Google Sheets API is also used to fetch and update admin data in real time.

5.5 Security and Data Handling

Security is ensured through controlled access where only authenticated university users can access the system. Data is securely stored in vector form inside ChromaDB, preventing unauthorized access to raw data. The system ensures that responses are generated only from verified university information.

5.6 User interface

Below figures represent the user interfaces and screens of AI campus Assistant. Log in Screen of System, **Error! Reference source not found.** Create account screen, **Error! Reference source not found.** Chat Screen of application, **Error! Reference source not found.** Comsats plus screen, **Error! Reference source not found.** course grade calculator screen, Figure 5.6 GPA calculator screen, Figure 5.7 CGPA calculator screen, Figure 5.8 GPA Planning screen, Figure 5.9 FAQs screen and Figure 5.10 Settings screen.

The Figure 5.1 shows the login interface of the AI Campus Assistant application. It allows users to sign in using their email address and password to access the system. The screen includes a “Sign In” button for authentication and a “Forgot Password” option for account recovery.

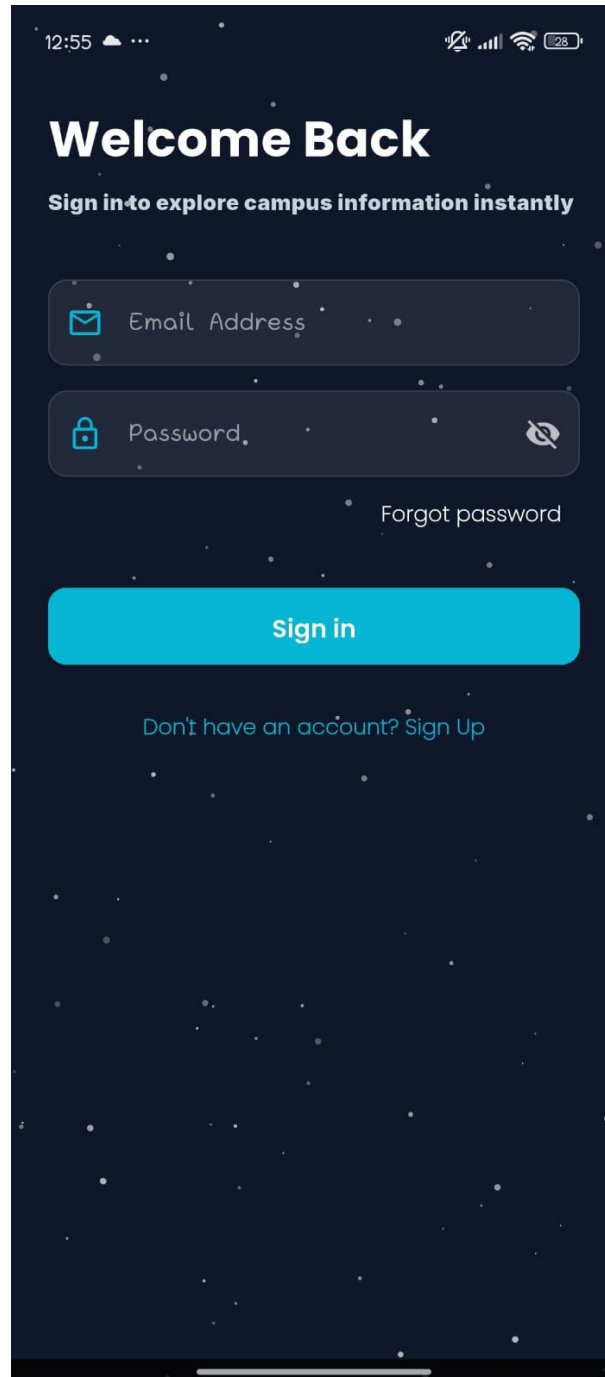


Figure 5-1 AI Campus Assistant Login screen

The Figure 5.2 shows the account registration interface of the AI Campus Assistant application. It allows new users to create an account by entering a username, university email address, and password. The screen includes a “Create Account” button to submit the registration details.

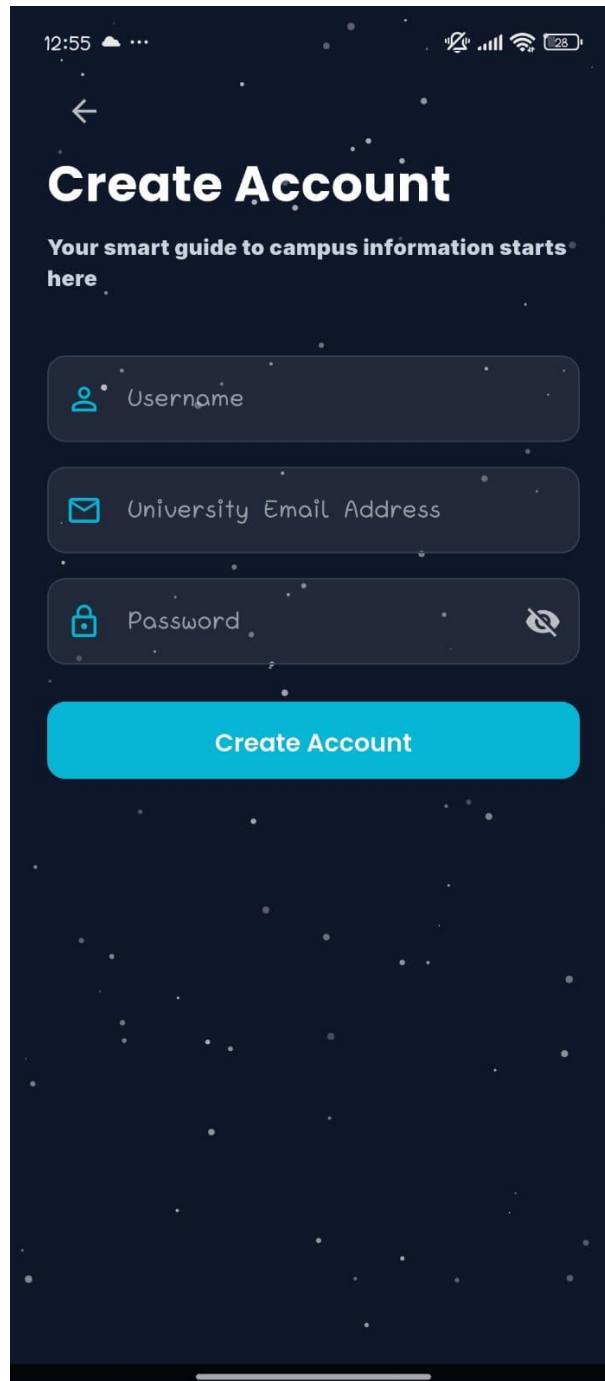
The image shows a mobile application screen for account creation. At the top, the status bar displays the time 12:55, signal strength, Wi-Fi, and battery level at 28%. Below the status bar is a dark blue header with a back arrow icon on the left and the title "Create Account" in white. Under the title is a subtitle: "Your smart guide to campus information starts here". The main content area contains three input fields, each with a light blue icon on the left: a person icon for "Username", an envelope icon for "University Email Address", and a lock icon for "Password". The password field has a small eye icon on the right to toggle visibility. Below the input fields is a large, rounded, light blue button with the text "Create Account" in white. The background of the screen is dark blue with a subtle pattern of white dots.

Figure 5-2 AI Campus Assistant Account Screen

The Figure 5.3 shows the chat screen of the AI Campus Assistant application. It allows users to interact with the system by asking questions in natural language just like ChatGPT but its only for university campus.

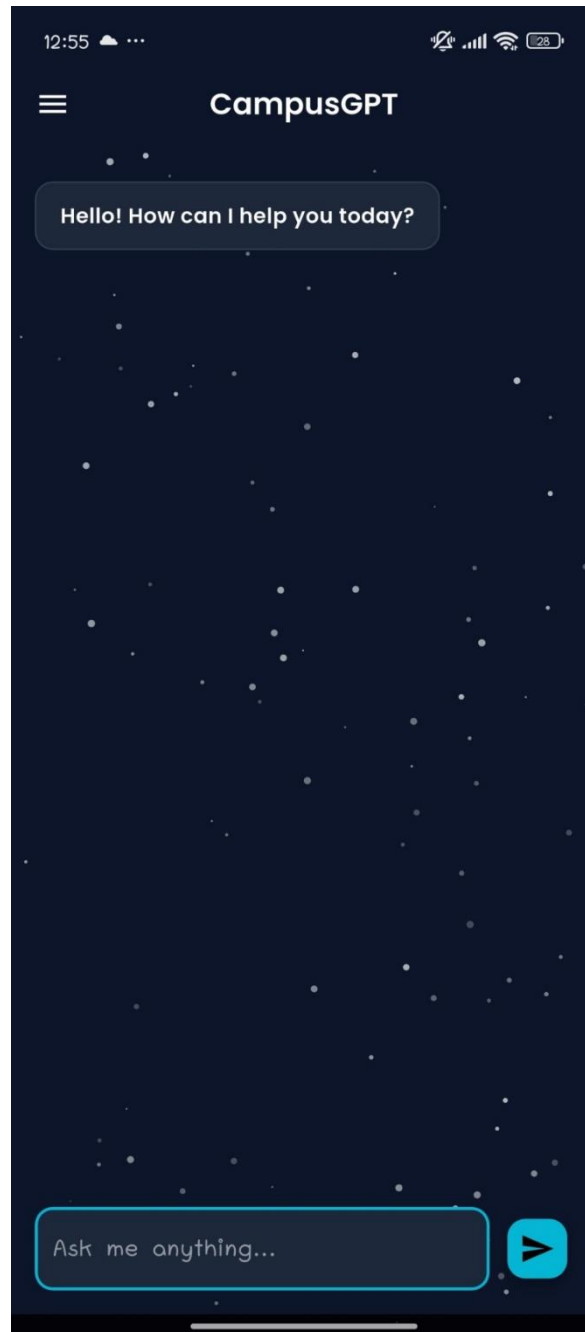


Figure 5-3 AI Campus Assistant Chat Screen

The Figure 5.4 shows the COMSATS Plus screen of the AI Campus Assistant application. It provides a centralized academic planning dashboard for students. The screen includes multiple tabs such as GPA, CGPA, and GPA Planning for easy navigation.

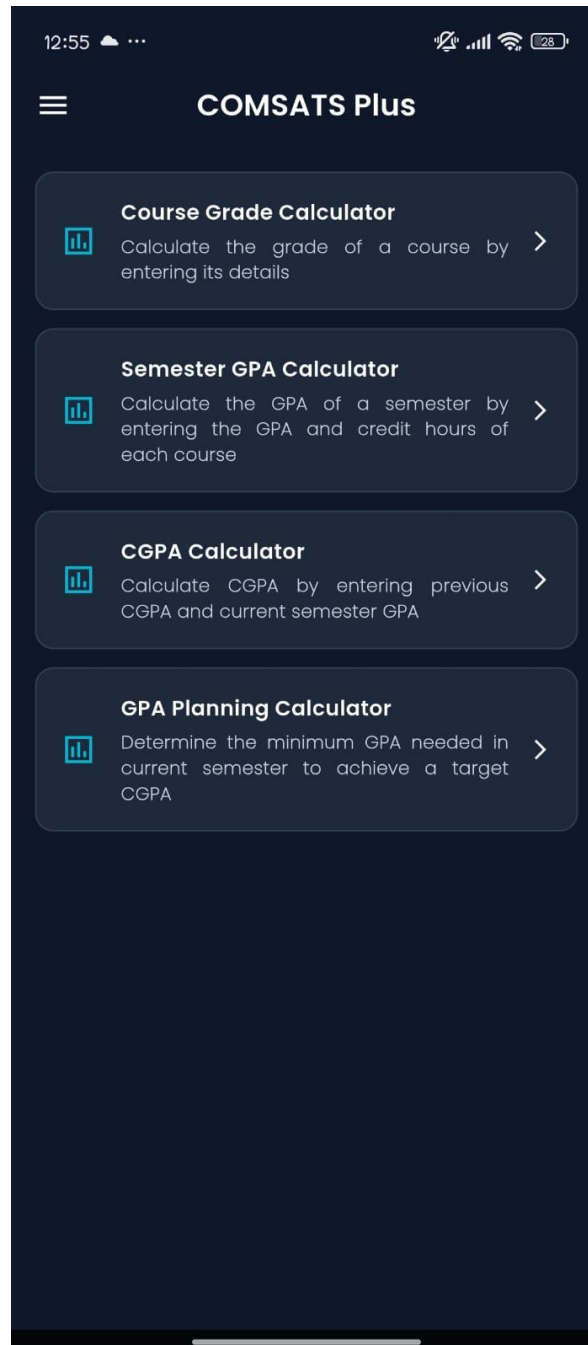


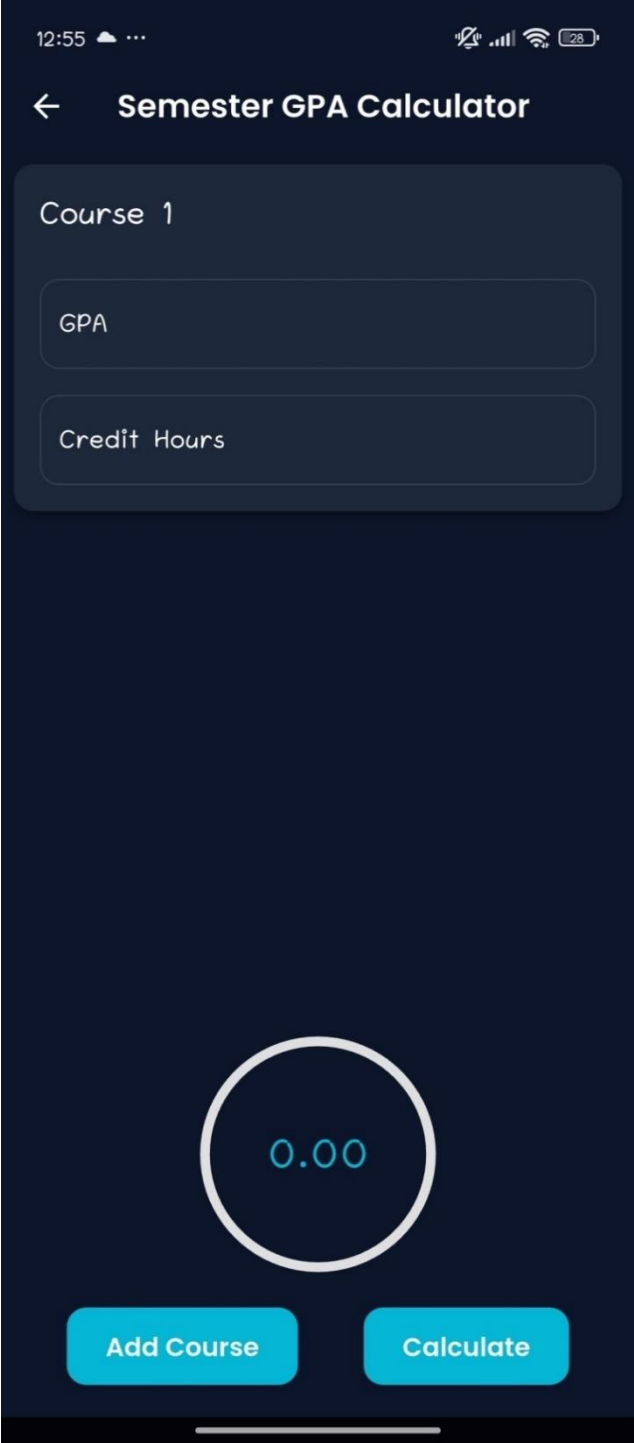
Figure 5-4 AI Campus Assistant Comsats plus Screen

The Figure 5.5 shows the course grade calculator interface of the AI Campus Assistant application. It allows students to calculate their expected grades by entering marks of assignments, quizzes, mid-term, and final exams.

The screenshot displays the 'Course Grade Calculator' screen. At the top, the status bar shows the time as 12:55 and various connectivity icons. Below the title bar, there is a back arrow and the title 'Course Grade Calculator'. The interface includes a 'Theory' button, followed by four dropdown menus for 'Quiz', 'Assignment', 'Midterm', and 'Final'. A checkbox labeled 'Has Lab' is positioned below these menus. At the bottom, there are two large circular displays showing '0.00' for 'GPA' and '0.00' for 'Marks'. A prominent blue 'Calculate' button is located at the very bottom of the screen.

Figure 5-5 AI Campus Assistant Course grade calculator Screen

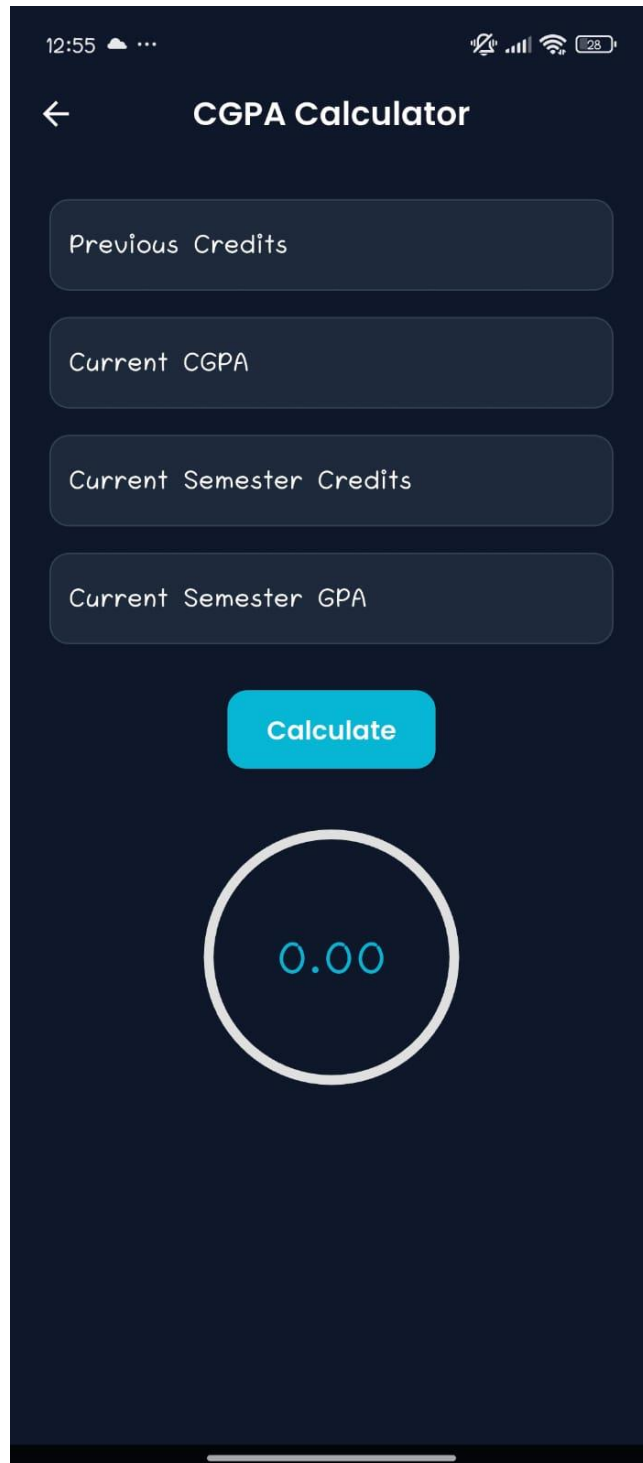
The Figure 5.6 shows the GPA interface of the AI Campus Assistant application. It allows students to view their semester GPA based on enrolled courses and earned grades.. Users can monitor their overall academic performance in one place.



The screenshot displays the 'Semester GPA Calculator' interface on a mobile device. At the top, the status bar shows the time as 12:55 and various connectivity icons. The app's header includes a back arrow and the title 'Semester GPA Calculator'. Below this, a section titled 'Course 1' contains two input fields: 'GPA' and 'Credit Hours'. A large circular display in the center shows the calculated GPA as '0.00'. At the bottom, there are two prominent blue buttons labeled 'Add Course' and 'Calculate'.

Figure 5-6 AI Campus Assistant semester GPA calculator Screen

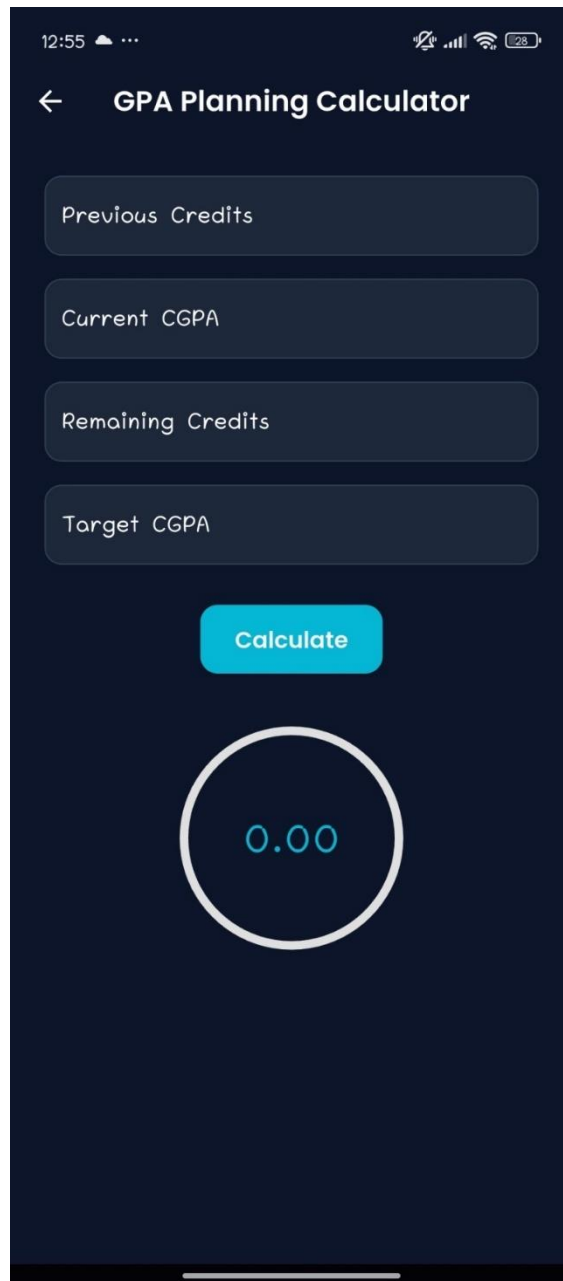
The Figure 5.7 shows the CGPA calculation interface of the AI Campus Assistant application. It allows students to view their cumulative grade point average based on all completed semesters. The system calculates CGPA by considering total credit hours and obtained grades.



The screenshot displays the 'CGPA Calculator' screen within a mobile application. At the top, the status bar shows the time as 12:55 and various connectivity icons. The app's header includes a back arrow and the title 'CGPA Calculator'. Below the header, there are four input fields with placeholder text: 'Previous Credits', 'Current CGPA', 'Current Semester Credits', and 'Current Semester GPA'. A prominent blue 'Calculate' button is positioned below these fields. At the bottom of the screen, a large white circle contains the text '0.00', representing the calculated CGPA. The entire interface is set against a dark blue background.

Figure 5-7 AI Campus Assistant CGPA calculator Screen

The Figure 5.8 shows the GPA planning calculator interface of the AI Campus Assistant application. It allows students to enter previous credits, current CGPA, remaining credits, and target CGPA.



The screenshot displays the 'GPA Planning Calculator' screen within a mobile application. At the top, the status bar shows the time as 12:55 and various connectivity icons. The app's header includes a back arrow and the title 'GPA Planning Calculator'. Below the header, there are four input fields, each with a light blue border and a dark blue background, labeled 'Previous Credits', 'Current CGPA', 'Remaining Credits', and 'Target CGPA'. A prominent orange 'Calculate' button is positioned below these fields. At the bottom of the screen, a large white circle contains the text '0.00' in orange, representing the calculated result. The entire interface is set against a dark blue background.

Figure 5-8 AI Campus Assistant GPA planning Screen

The Figure 5.9 shows the frequently asked questions (FAQs) interface of the AI Campus Assistant application. It provides pre-defined answers to common queries related to university admissions, exams, fees, and campus services.

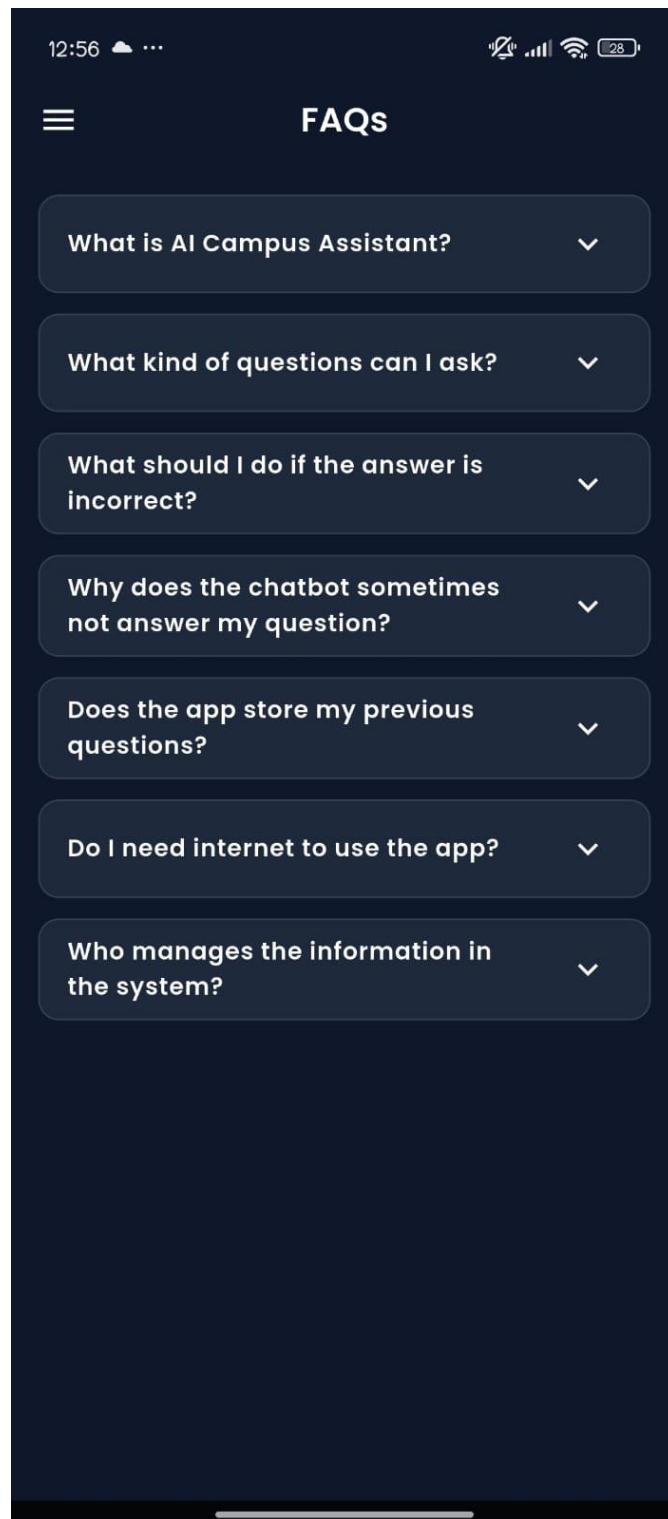


Figure 5-9 AI Campus Assistant FAQs Screen

The Figure 5.10 shows the settings interface of the AI Campus Assistant application. It allows users to manage their personal preferences and application configurations. The screen includes options such as profile update, password change, and notification settings.

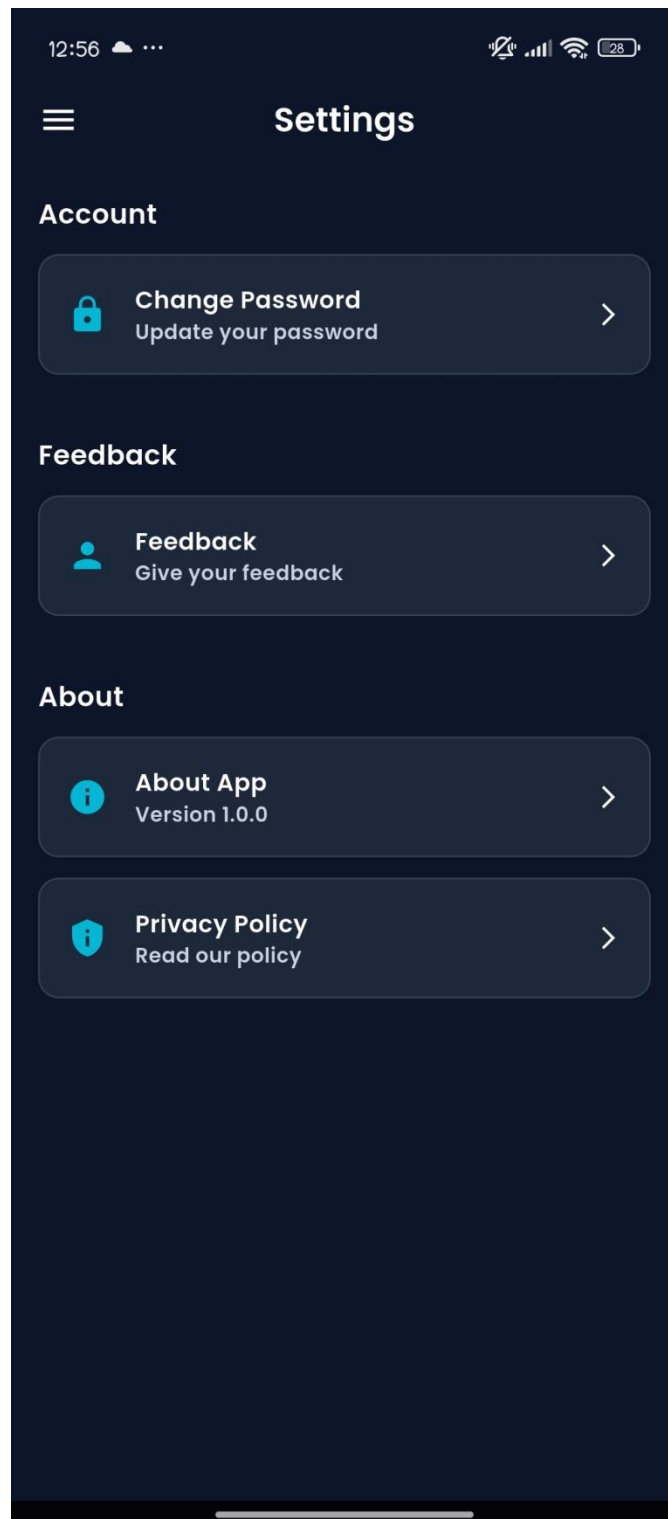


Figure 5-10 AI Campus Assistant Settings Screen

Chapter 6

Testing and Evaluation

6 Testing

6.1 Unit Testing

Unit tests is conducted to assess the individual components of the AI campus Assistant.

Table 6.1 Unit Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	User Authentication	Valid university email login	User successfully logs into the system	Pass
UT-002	Ask Query	User inputs a valid query	Query is successfully submitted	Pass
UT-003	Blank Query Field	Empty query submission	System shows error message	Pass
UT-004	Retrieve Documents	Query sent to vector DB	Relevant chunks are retrieved	Pass
UT-005	RAG Response Generation	Query + retrieved chunks	Accurate response generated by LLM	Pass
UT-006	Feedback Submission	User submits feedback	Feedback is stored successfully	Pass
UT-007	Chunk Generation	Convert data into chunks	Data is properly chunked	Pass
UT-008	Admin Upload Data	Upload/update Google Sheets data	Data is successfully updated	Pass

6.2 Manual Testing

During the manual testing phase, we rigorously assessed "AI campus Assistant" for various functionalities. Any issues identified during testing were promptly addressed to guarantee a seamless user experience within the application.

Table 6.2 Manual Testing

Test Cases	Expected Result	Actual Result	Pass/Fail
App Launch	Successful	Successful	Pass
User Login	Successful	Successful	Pass
User Login	Successful	Successful	Pass
Chat Interface Display	Accurate	Accurate	Pass
Ask Query	Accurate	Accurate	Pass
Receive Response	Accurate	Accurate	Pass
Query History	Accurate	Accurate	Pass
Admin Data Update	Successful	Successful	Pass
Manage Profile	Successful	Successful	Pass
App Performance and Response	Smooth	Smooth	Pass
App Navigation	Easy	Easy	Pass
App Usability and User Friendliness	High	High	Pass
Logout	Successful	Successful	Pass

6.3 Functional Testing

Functional testing verifies End to end functionalities of a AI campus Assistant against its functional requirements and specifications, to ensure that the software behaves as expected.

Table 6.1 Functional Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
FT-001	User Authentication	Login with valid university email	User successfully logs in	Pass
Ft-002	Ask Query	User enters a question	Query is processed correctly	Pass
FT-003	RAG Pipeline	Query retrieval + generation	Accurate response generated	Pass
FT-004	Document Retrieval	Fetch relevant chunks from DB	Relevant documents retrieved	Pass
FT-005	Feedback System	Submit feedback	Feedback successfully stored	Pass

FT-006	Admin Data Management	Upload/update data	Data updated successfully	Pass
FT-007	Password Encryption	Password encryption	Passwords are securely encrypted a	Pass

6.4 Integration Testing

Integration testing is a software testing technique that focuses on evaluating the interactions between different software modules or components of AI campus Assistant.

Table 6.2 Integration testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
IT-001	Frontend–Backend Integration	API request from UI to FastAPI	Successful communication	Pass
IT-002	Backend–Vector DB Integration	Query sent to ChromaDB	Relevant data retrieved	Pass
IT-003	Full RAG Pipeline	Query → Retrieval → Response	Accurate end-to-end output	Pass

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

The **AI Campus Assistant** project represents a significant advancement in improving access to university-related information through the use of intelligent technologies. The system successfully addresses the common challenges faced by students, faculty, and visitors in obtaining accurate and timely information.

By integrating **Artificial Intelligence techniques such as Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs)**, the system provides accurate, context-aware, and reliable responses based on verified university data. Unlike traditional methods, the AI Campus Assistant offers a centralized and efficient platform where users can ask queries and receive instant answers through a user-friendly chat interface.

The use of structured data management through Google Sheets, along with vector databases such as ChromaDB, ensures that the information remains up-to-date and easily maintainable by administrators. Additionally, the system's modular architecture, including frontend, backend, and AI components, enhances scalability and maintainability.

Overall, the AI Campus Assistant has successfully achieved its objectives by providing a smart, efficient, and reliable solution for handling university-related queries. It improves user experience, reduces dependency on manual processes, and enhances communication within the university environment.

7.2 Future Work

Although the **AI Campus Assistant** has been successfully developed, there are several opportunities for future enhancements and improvements:

- **Multilingual Support:**

Continuously adding new features and capabilities to meet evolving user needs, such as voice call and media sharing,

- **Enhanced Security:**

Strengthening the security features to ensure the protection of user data and privacy

- **Advanced Personalization:**

Providing personalized responses based on user preferences, history, or academic profile.

- **User Feedback and Iteration:**

Regularly gathering user feedback and using it to make iterative improvements to the platform.

- **Mobile Application Development:**

Further improving the mobile application for better performance and user experience across Android and iOS platforms.

- **User Training and Support:**

Expanding user training materials and support channels to help users make the most of the platform. Incorporating these future work elements will help AI campus Assistant stay current, relevant, and effective in catering to the dynamic needs of the university community while maintaining a high standard of security and usability.

References

- [1] M.Mehra, M. Kumar, M. Maurya, A. Sharma, C., & Shanu. (2021). App Development. Annals of the Romanian Society for Cell Biology.
- [2] van Vliet, H. (2007). Software Engineering: Principles and Practice. Wiley.
- [3] Cockburn, A. (2006). Agile Software Development (Second Edition).
- [4] Bāk, K. (2022, April 8). Real-time Chat Application Design: Architecture and Workflow. Medium.[Online].
- [5] Roman, G. (2020, March 22). What is a Process Flow Diagram. Lucidchart. [Online].
- [6] Purchase, H. C., Colpoys, L., McGill, M. J., & Carrington, D. A. (2001, December). UML Class Diagram Syntax: An Empirical Study of Comprehension. In Australasian Symposium on Information Visualisation, InVis.au, Sydney, Australia.
- [7] Ahmed, S., & Mahmood, Q. (2019). An authentication based scheme for applications using JSON web token. 2019 22nd International Multitopic Conference (INMIC), Islamabad, Pakistan.